

LAB 3: Continuous Training – Alejandro Sosa Corral

Preguntas:

Pregunta 1:

En este caso sí, se obtendrían los mismos resultados, debido principalmente la semilla utilizada en el proceso. En primer lugar, la semilla se utiliza para dividir los datos de training y test, y, por otro lado, se utilizan al definir la Regresión Logística para que siempre se ejecute de la misma manera.

Esta semilla se define mediante el argumento *random_state* tanto en *train_test_split* como en *LogisticRegression*.

Pregunta 2:

Si lanzáramos una petición de entrenamiento a la API con datos que rompan alguna regla (sea por cantidad de muestras o clases), el sistema está programado para devolver un error HTTP 422 describiendo la regla que se ha incumplido, por lo que el modelo ni se entrenaría ni se activaría. Esta restricción tiene sentido ya que no queremos entrenar modelos con conjuntos de datos que reduzcan en gran medida la calidad del resultado.

Pregunta 3:

El motivo por el que registramos en el historial todos los modelos probados, incluyendo aquellos que no suponen una mejora, es para mantener una trazabilidad que nos permita observar no solo la accuracy de estos modelos sino el motivo por el que estos fueron rechazados, dándonos información que podría sernos útil para futuros modelos.

Pregunta 4:

En mi opinión, en esta clase de modelos es completamente acertado hacer uso de $\geq$ a la hora de decidir si un modelo reemplaza a otro. Incluso aunque un modelo obtenga la misma accuracy que otro, el hecho de que haya sido entrenado con un número mayor de muestras que el anterior le da una mayor robustez y mayor calidad de generación que el modelo entrenado con menos muestras. Además, eventualmente, el crecimiento del modelo dará lugar a que la accuracy se estabilice en su punto más alto y deje de subir, por lo que un $>$ estricto haría que el modelo deje de evolucionar.

Reflexión Programación:

Para los tres escenarios hipotéticos presentados en el ejercicio:

Escenario 1: En este primer caso, debido a que los falsos negativos son mucho más caros que los falsos positivos, es esencial que centremos el entrenamiento de nuestro modelo en asegurar que no se nos escape ningún fraude. Por ello, deberíamos aplicar una política que se centre en la mejora de la clase negativa (`per_class_f1`).

Escenario 2: En este segundo caso, el coste de error es mucho más bajo, por ello, no tenemos que ser tan estrictos como antes a la hora de aceptar cambios. Nada más detectemos un modelo que suponga mejora en la `accuracy`, por pequeña que sea, deberíamos aceptarlo como reemplazo, aplicando una política más laxa (`any_improvement`).

Escenario 3: Por último, en este caso lo más importante de nuestro sistema es la estabilidad y previsibilidad del comportamiento de nuestro modelo, no pudiendo arriesgar a reemplazarlo al detectar cambios pequeños en la `accuracy`. Por ello, aplicaríamos una política de mejora más estricta que únicamente considere reemplazar el modelo si este supone una mejora significativa en la `accuracy` de nuestras predicciones (`min_delta`).

(Resto de ejercicios y pseudocódigo en resto de entregable)